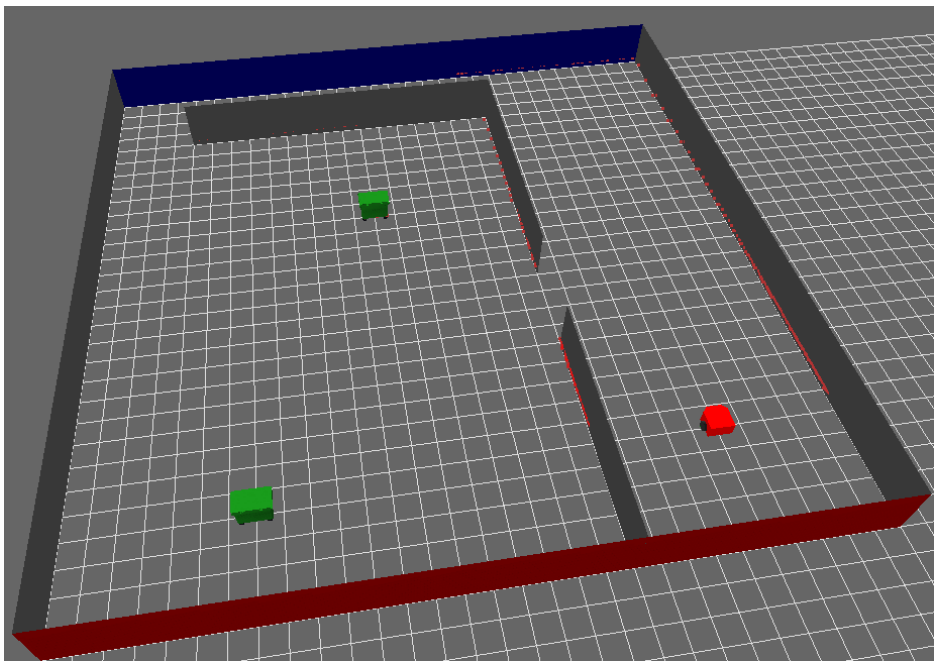
Programación de tareas en robots móviles

A lo largo de esta práctica se va a desarrollar una serie de nodos de ros2 los cuales van a implementar diversas tareas para el control de un robot móvil.

El entorno en el cual se va a mover el robot se da predefinido:



Se hará uso de la biblioteca “Yasmin”, la cual sirve para crear y utilizar máquinas de estados finitos.

Por otro lado, también se hará uso de un nodo para la detección de los colores percibidos por la cámara del robot.

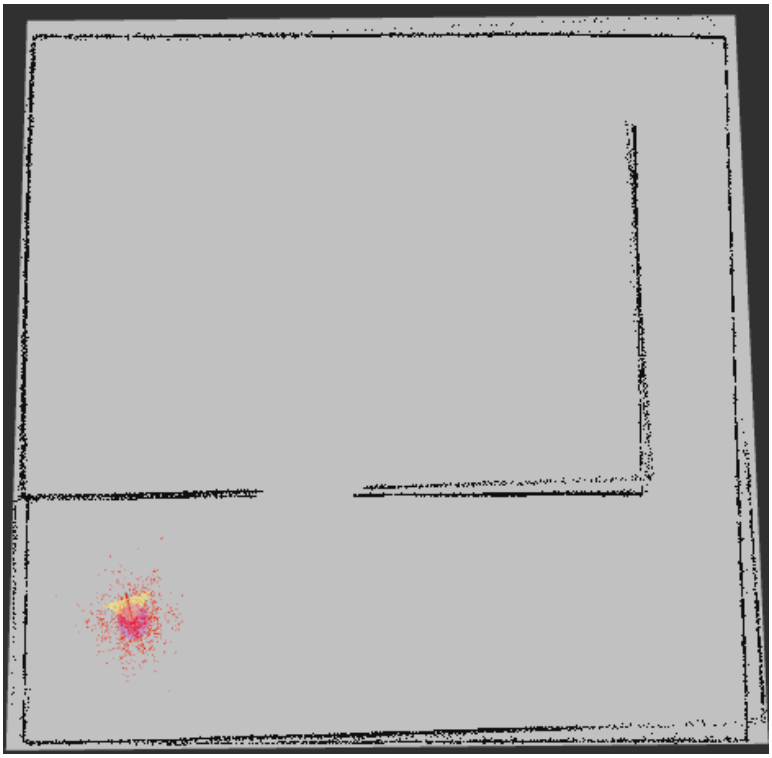
Cabe remarcar que el mapa del entorno ha sido modificado y se ha eliminado el contenedor que había situado en el pasillo. Se ha hecho con el fin de poder mostrar un recorrido más largo, ya que cuando estaba este objeto las simulaciones dejaban poco recorrido por su detección temprana.

Tarea a desarrollar:

El problema a resolver de forma general, es la gestión de tareas de movimiento en un robot movil según ciertos imputs del entorno y junto a su programación básica de movimiento. En este caso se especifica que el robot vaya deambulando por el espacio

mientras que no encuentre un objeto de color verde, cuando esto ocurra debe volver a la posición inicial.

Para poder lograr este último paso se hace uso del generador de trayectorias y de un mapa de la zona en el cual solo vienen definidadas las paredes, los objetos no están incluidos.



En el mapa también se puede ver la posición estimada del robot mediante el AMCL que incorpora el sisema de nav2.

Se han realizado las siguientes pruebas de movimiento según diferentes casos. Para todas ellas se ha utilizado además un detector de color el cual publica en el topic */color_detected* el número de píxeles que ocupa el objeto detectado .

Casos de movimiento:

Caso de movimiento 1: FSM condicional con movimiento aleatorio

En este caso el robot va a seguir una trayectoria recta hasta que se encuentre con una pared o un obstáculo, en ese caso, girará hacia donde más espacio tenga disponible, así evitará el problema por bloqueo en las esquinas donde solía atascarse.

Además volverá a su posición “Home” cuando el objeto se detecte de forma significativa, es decir, cuando el número de píxeles que ocupa sea mayor que cierto

umbral, lo cual dejará claro que está suficientemente cerca de él. Esta acción se mediante el generador de trayectorias.

Se hace una ejecución de una FSM con 2 estados: movimiento libre y vuelta a HOME.

https://youtu.be/ClKOCwsvArU

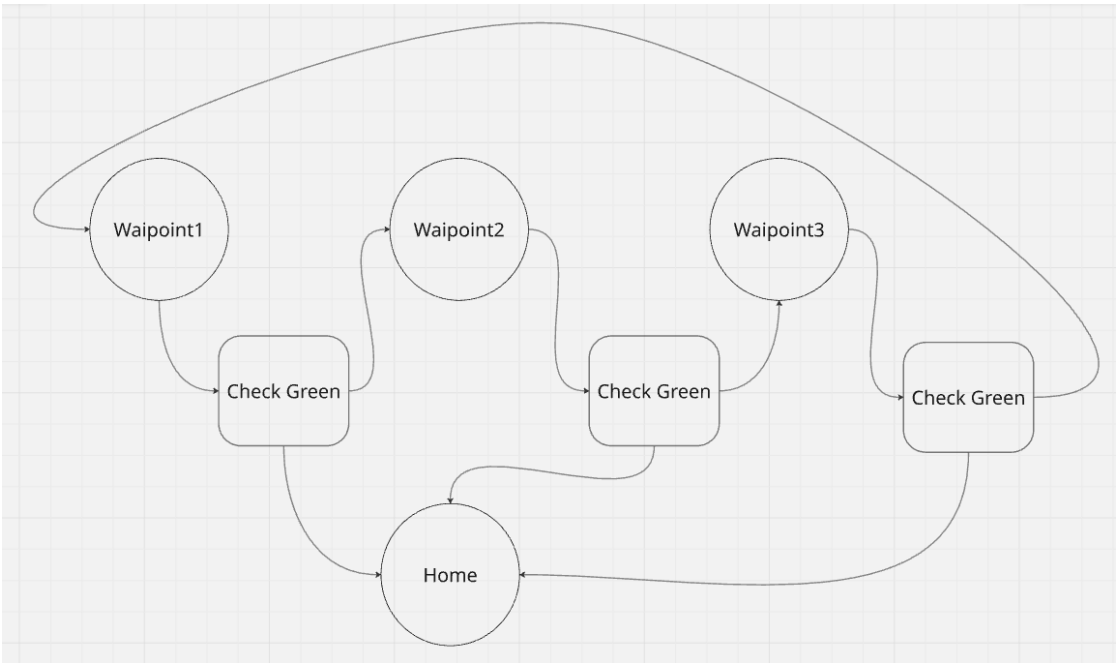
Caso de movimiento 2: FSM secuencial con movimiento por waypoints

El robot va a ir moviendose por el espacio siguiendo una serie de waypoints (se quedará en modo vigilancia) hasta que la cámara detecte un objeto de color verde, en este mapa son los contenedores. Al hacerse la detección el robot volverá a su posición inicial.

Para poder llegar hasta los objetivos se hace uso de la navegación que se incorpora con el comando:

```
ros2 launch nav2_bringup bringup_launch.py map:=./world/prac3.yaml
```

Esta implementación será mediante una máquina de estados finitas y tendrá el siguiente esquema:



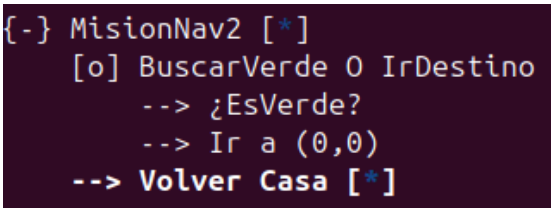
Si durante el trayecto se ha detectado algún objeto de color verde el robot volverá a su posición inicial (HOME) después de haber completado el trayecto hasta el waypoint actual. Al igual que en el caso anterior se dirá que ha detectado un objeto cuando este supere un cierto umbral.

https://youtu.be/9_pmFezJrOc

Caso de movimiento 3: Implementación con behavior trees

En este último caso se hace una implemetación mediante los behaviors trees, se desarrolla la tarea basica (movimiento por el espacio hasta la detección del color verde), esta vez se aumenta más la tolerancia hasta 7000 píxeles para volver al HOME.

El esquema que sigue el árbol de comportamiento es el siguiente:



https://youtu.be/pS_ssN-l0mw

Con respecto a una FSM se produce una mejora respecto a la filosofía del control. Haciendo uso de los BT se consideran las posibles situaciones con una determinada frecuencia y se empieza desde el nodo principal; para tomar la decisión de seguir con la tarea o parar para realizar una nueva porque las condiciones han cambiado.

En el caso de la FSM, cuando se entra en un estado se “secuestra” la toma de deciciones hasta que no concluye (a no se que se ponga un validador para cancelar en bucle).

Posibles fallos y mejoras

La velocidad del robot máxima que viene predefinida mediante el navigator es lenta, se ha intentado aumentar pero surgían fallos o velocidades demasiado altas para el entorno.

Además está el problema de ajustar el valor al cual el generador de trayectorias permite o no el paso. Es decir, si pasa por un hueco demasiado estrecho y el factor de seguridad es demasiado restrictivo.

Por lo general, el generador de trayectorias funciona correctamente, pero en casos donde tenga que ir en sentido contrario hacia donde está orientado, la trayectoria generada incluye ir hacia detrás, se podría ajustar de mejor forma haciendo un arco hacia delante.